UNIVERSIDAD PANAMERICANA

Facultad de Ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas

Lenguajes Formales y Autómatas

Hipstagram

Documento de Diseño de Arquitectura

Kevin Eliazar Herrera Alburez (000113637)

Carlos Enrrique Otoniel Bautista Sarazúa (000133047)

Herbert David Estrada Trujillo ()

Docente: Ing. Mario Morales

Chimaltenango, Chimaltenango, marzo de 2,026

Hipstagram

Documento de Diseño de Arquitectura

Kevin Eliazar Herrera Alburez (000113637)

Carlos Enrrique Otoniel Bautista Sarazúa (000133047)

Herbert David Estrada Trujillo

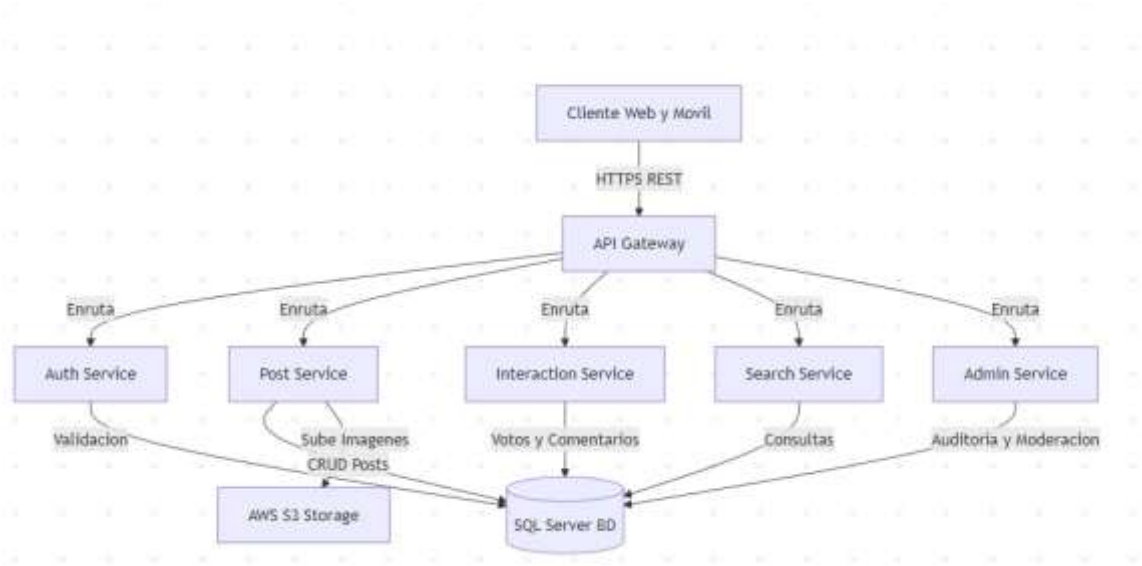
Docente: Ing. Mario Morales

Chimaltenango, Chimaltenango, noviembre de 2,026

1. Introducción

EL proyecto de Hipstagram consiste en el diseño e implementación de una plataforma tipo red social orientada a la publicación y descubrimiento de fotografías, basada en una arquitectura de microservicios, con mecanismos robustos de seguridad, auditoria, moderación de contenido y despliegue continuo en la nube. El proyecto busca el aprendizaje sobre el manejo y adquisición de los servicios de la nube, durante la primera fase del proyecto el objetivo es establecer las bases formales, técnicas y organizativas del proyecto, incluyendo la planificación estructurada, definición clara de requerimientos, modelo de base de datos, metodología aplicada, configuración de versiones git y ambiente base con Docker Compose. La fase descrita es crítica debido a que la dirección técnica y metodología que guiara las siguientes 16 semanas de desarrollo del proyecto.

2. Diagrama de Componentes



3. Descripción de Componentes

3.1. Cliente Web y Móvil: Las interfaces de usuario final (desarrolladas posiblemente en React o Flutter) que consumen las APIs del sistema.

3.2. API Gateway: Es el punto de entrada único al sistema. Se encarga de recibir las peticiones HTTP, validar la firma de los tokens JWT y enrutar el tráfico hacia el microservicio correspondiente.

3.3. Auth Service: Microservicio encargado de la seguridad. Gestiona el registro, validación de credenciales (hashing), inicio de sesión y emisión de los tokens JWT con los roles de usuario.

3.4. Post Service: Encargado de la lógica de publicaciones. Valida la longitud de la descripción (máximo 128 caracteres), extrae hashtags y guarda el archivo binario de la foto en la nube.

3.5. Interaction Service: Gestiona la lógica de las interacciones sociales, específicamente los comentarios paginados y el sistema de votos (garantizando un único voto por usuario por publicación).

3.6. Search Service: Componente optimizado para búsquedas. Permite encontrar contenido mediante filtrado por hashtags y búsqueda por texto libre, además de generar el feed ordenado por "likes".

3.7. Base de Datos (SQL Server): Almacenamiento relacional centralizado, estructurado con estrictas reglas de integridad referencial y seguridad (roles de base de datos).

4. Diagrama de capas.

5. Descripción de la seguridad y credenciales

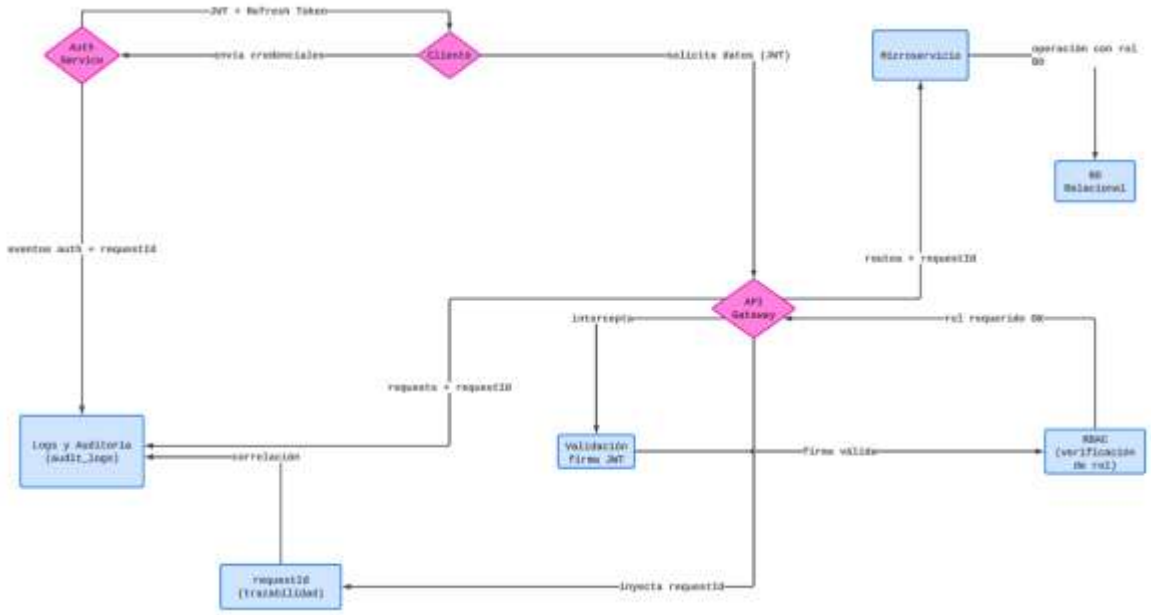
5.1. Autenticación

Se brindará control de acceso mediante la imposición de roles dentro de la base de datos, ningún usuario podrá disponer de que rol tomar, será únicamente el administrados quien designe los roles en la creación de un nuevo usuario. Se tiene contemplados dos tipos de usuario, USER y ADMIN. El API valida que solo el rol de Admin tenga acceso a endpoints críticos como log de auditar y gestión de black list de palabras prohibidas.

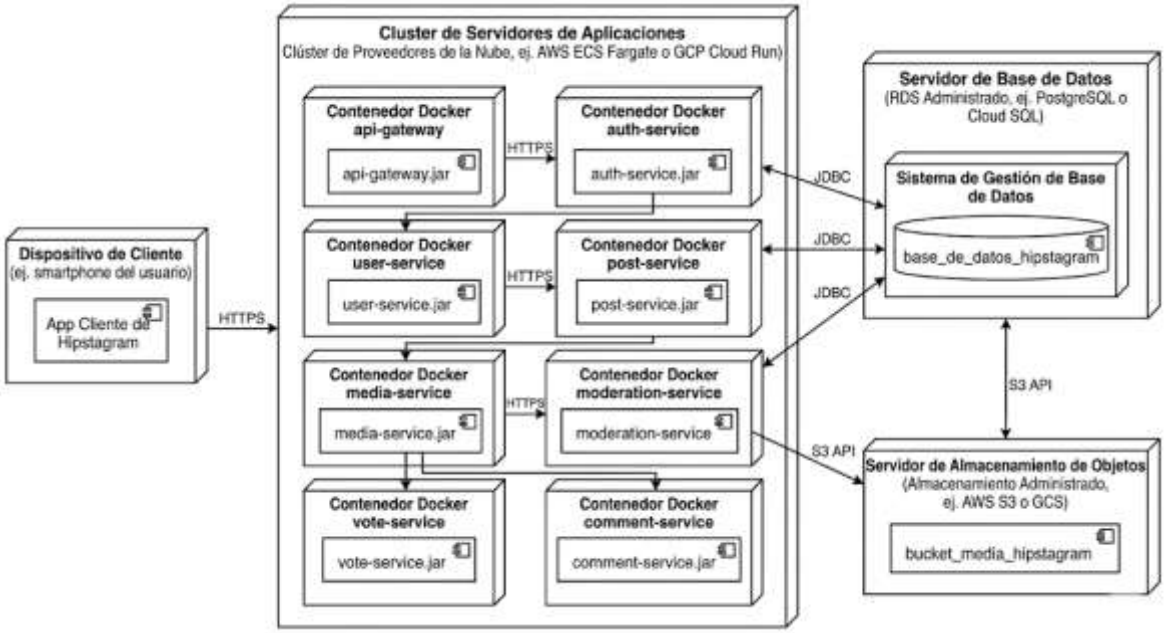
5.2. Seguridad de la base de datos

Siguiendo con el principio de menor privilegio, se segmentan los accesos a nivel primario en la base de datos. Habrá un usuario que será utilizado por los microservicios para operaciones comunes, típicas de un crud. Habrá un usuario admin que tendrá permisos para todo incluyendo acceso a logs.

6. Diagramas de seguridad



7. Vistas de la implementación



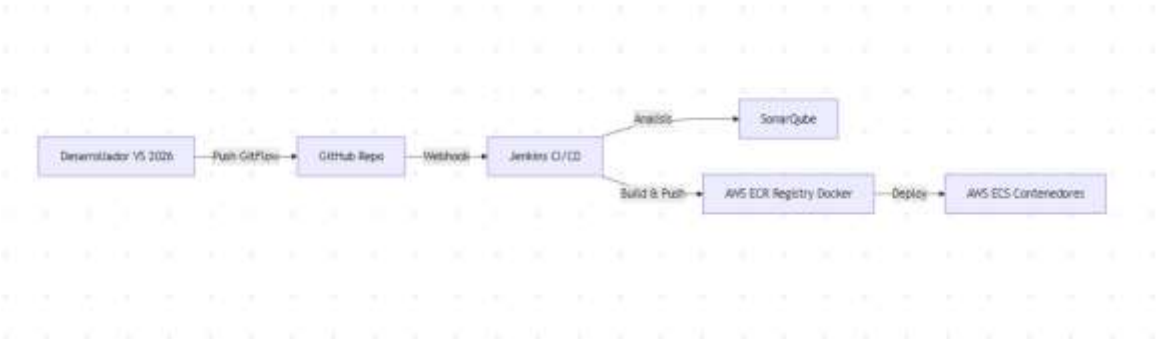
8. Descripción de la implementación

8.1. Dispositivo de Cliente: Este sería la computadora o el teléfono que usaría el usuario que estaría usando la aplicación.

8.2. Servidor de Aplicaciones: Es un clúster gestionado que administra los contenedores de Docker de los microservicios.

8.3. Servidor de Base de Datos: Un servicio de base de datos administrado que garantiza alta disponibilidad y copias de seguridad de los esquemas de datos.
8.4. Servidor de almacenamiento de Objetos: Un servicio especializado para almacenar físicamente las fotografías.
8.5. Protocolos y Conexiones:
    8.5.1. HTTPS: Protocolo seguro para toda la comunicación entre el cliente, el API y los microservicios.
    8.5.2. JDBC: Protocolo estándar para que los microservicios ejecuten consultas en SQL, pero en el servidor de base de datos.
    8.5.3. API S3: Protocolo para ejecutar el almacenamiento de datos.
8.6. Artefactos y Contenedores:
    8.6.1. App Cliente de “Hipstagram”: Parte del Frontend en la que interactúan directamente los usuarios.
    8.6.2. Contenedor Docker: Cada microservicio autónomo está empaquetado en su propia imagen de Docker para un aislamiento total.

9. Diagrama de la implementación



10. Vista puesta en marca.
    10.1. El punto de entrada será un Application Load Balancer (ALB) conectado al API Gateway.
    10.2. Los microservicios en C# se ejecutarán en **AWS ECS (Fargate)**, garantizando escalabilidad automática sin necesidad de gestionar servidores físicos.
    10.3. La persistencia de datos relacional estará alojada en **AWS RDS (SQL Server 2022)**, configurada con respaldos automáticos y Multi-AZ para alta disponibilidad.
    10.4. Los archivos multimedia (las fotografías publicadas por los usuarios) se almacenarán de forma directa y segura en **Amazon S3**.
11. Vista de Datos
    La persistencia de la información garantiza la consistencia transaccional y la integridad referencial. Se han configurado restricciones jerárquicas estrictas (ON DELETE NO ACTION) para evitar la eliminación de registros "padre" (ej. Usuarios) si poseen registros dependientes (ej. Publicaciones o Votos).

Adicionalmente, se utilizan transacciones (BEGIN TRAN, COMMIT, ROLLBACK) para operaciones críticas en la base de datos y evitar datos huérfanos.

12. Diagrama de entidad relación

13.Diccionario de datos

13.1. Tabla Roles

- IdRol (INT, PK): Identificador único autoincremental del rol.
- NombreRol (NVARCHAR 30, UNIQUE, NOT NULL): Nombre del rol (Admin, Usuario).

- Descripcion (NVARCHAR 100): Detalles de los permisos del rol.
- Activo (BIT): Bandera lógica de habilitación (1=Activo, 0=Inactivo).

13.2. **Tabla Usuarios**

- IdUsuario (INT, PK): Identificador único del usuario.
- Usuario (NVARCHAR 50, UNIQUE, NOT NULL): Nombre de usuario para login.
- PasswordHash (NVARCHAR 255, NOT NULL): Contraseña encriptada.
- IdRol (INT, FK, NOT NULL): Llave foránea hacia la tabla Roles.
- Activo (BIT): Bandera lógica para bloqueo de usuarios.
- FechaCreacion (DATETIME): Fecha de registro en el sistema.

13.3. **Tabla Publicaciones**

IdPublicacion (INT, PK): ID único de la foto.

IdUsuario (INT, FK, NOT NULL): ID del autor.

UrlImagen (TEXT, NOT NULL): Ruta del archivo almacenado en AWS S3.

Descripcion (VARCHAR 128): Texto corto de la publicación.

Estado (VARCHAR 20): Estado de moderación (OK o BLOQUEADO).

13.4. **Tabla Votos**

13.5. IdUsuario (INT, PK/FK): ID del usuario que vota.

13.6. IdPublicacion (INT, PK/FK): ID del post votado. *(Forma Llave Primaria Compuesta)*.

13.7. TipoReaccion (VARCHAR 10): Tipo de voto (LIKE o DISLIKE).

13.5. Tabla LogsAuditoria

13.6. Tabla aislada (sin FKs para garantizar inmutabilidad) que registra campos como: FechaHora, IdUsuarioActor, Accion, IdEntidad, Resultado e IpOrigen. Visible únicamente por administradores.